ช่วงบ่าย: ลงมือทำ

เห็นข้างใน LLM และ VAE ด้วยภาพและการเปรียบเทียบ

ดร.ธีรพงศ์ อรชร



ภาพรวม

บ่ายนี้ทำอะไร และเชื่อมกับตอนเช้าตรงไหน

ในห้อง

Notebook 1 · Lab 1–3

เปิดฝากระโปรง LLM ภาษาไทย

- Tokenizer & Embedding
- LLM เขียนทีละ token, Attention
- Base vs Instruction, ความจำ, Hallucination
- Temperature & Top-p

↔ ส่วนที่ 3–4 ตอนเช้า

ในห้อง

Notebook 2 · Lab 4

VAE vs Autoencoder

- เทรน 2 โมเดลด้วยโค้ดเดียวกัน
- เทียบ latent space
- เทียบการสุ่มสร้างภาพใหม่

↔ ส่วนที่ 2 ตอนเช้า

การบ้าน

Notebook 3

ระบบวิเคราะห์รีวิว

- รีวิวภาษาไทย: WangchanBERTa + ThaiNER
- LLM ตอบ JSON + Pydantic ตรวจ
- กราฟสรุป + เทียบโมเดลเฉพาะงาน vs LLM

↔ ส่วนที่ 5 ตอนเช้า

Lab 0

เตรียม Google Colab

1. **เปิดไฟล์ .ipynb**
   File → Upload notebook หรือเปิดจาก Google Drive

2. **เปิด GPU**
   Runtime → Change runtime type → T4 GPU → Save

3. **รันเซลล์แรก**
   Shift+Enter ต้องเห็น "ใช้ GPU: Tesla T4"

4. **โหลดโมเดลทันที**
   ใช้ 3–5 นาที (Typhoon2 ขนาด 1.2B สองตัว) ระหว่างรอ ฟังอธิบาย Lab 1

⚠ ต่างจากแล็บ IBM

ตัด %pip install torch==...+cpu ออก เพราะจะทับ PyTorch เวอร์ชัน GPU ของ Colab

เซลล์แรกติดตั้ง pythainlp และฟอนต์ไทย (Sarabun) ให้กราฟแสดงภาษาไทยได้

attn_implementation="eager" เพื่อดึงค่า attention มาวาดภาพ

Lab 0

โมเดลภาษาไทยที่ใช้: โครงคล้ายกัน แต่ฝึกมาต่างกัน

ชื่อในแล็บ	ตระกูล (ตอนเช้า)	ฝึกมาอย่างไร	จะเห็นพฤติกรรม
Typhoon-Base	Decoder-only	pretrain อย่างเดียว = base model	เขียนต่อ ไม่ตอบคำถาม
Typhoon-Instruct	Decoder-only	ฝึกต่อด้วย instruction + บทสนทนา	ทำตามคำสั่ง คุยได้
WangchanBERTa	Encoder-only	pretrain ภาษาไทย + fine-tune sentiment	attention สองทาง · tokenizer ไทย
GPT-2 (tokenizer)	—	ภาษาอังกฤษ	ใช้เทียบว่าหั่นภาษาไทยอย่างไร

Typhoon2 (SCB 10X) ต่อยอดจาก Llama 3.2 · WangchanBERTa (VISTEC-depa) · Base กับ Instruct คือโมเดลเดียวกัน ต่างแค่การฝึกขั้นหลัง

LAB 1

Tokenizer & Embedding

จากตอนเช้า: ขั้น ① ② ของ Transformer — ข้อความ → token → token id → vector

เป้าหมาย

- เทียบ tokenizer อังกฤษ vs ไทย ด้วย token สี
- วัดว่าแต่ละ tokenizer ใช้ token กับภาษาไทยมากน้อยแค่ไหน
- วาด embedding ของคำไทยให้เห็นกลุ่มคำ

Lab 1

ข้อความ → Token → Token ID



notebook แสดงแต่ละ token เป็นแถบสี เทียบ GPT-2 · WangchanBERTa · Typhoon2 บนประโยคภาษาไทยเดียวกัน

โค้ด: เทียบ tokenizer และนับ token

```
def pieces_of(tokenizer, text):
    ids = tokenizer.encode(text, add_special_tokens=False)
    return ids, [tokenizer.decode([i]) for i in ids]

for name, tk in TOKENIZERS.items():       # 3 tokenizer
    ids, pieces = pieces_of(tk, text)     # → แถบสีต่อ token

counts[name]["ไทย"] += len(tk.encode(th))   # นับ token 3 คู่ประโยค
ax.bar(...)                          # กราฟแท่ง EN vs TH
```

encode()

ข้อความ → เลข id (ปิด special tokens เพื่อดูเฉพาะเนื้อความ)

decode([i])

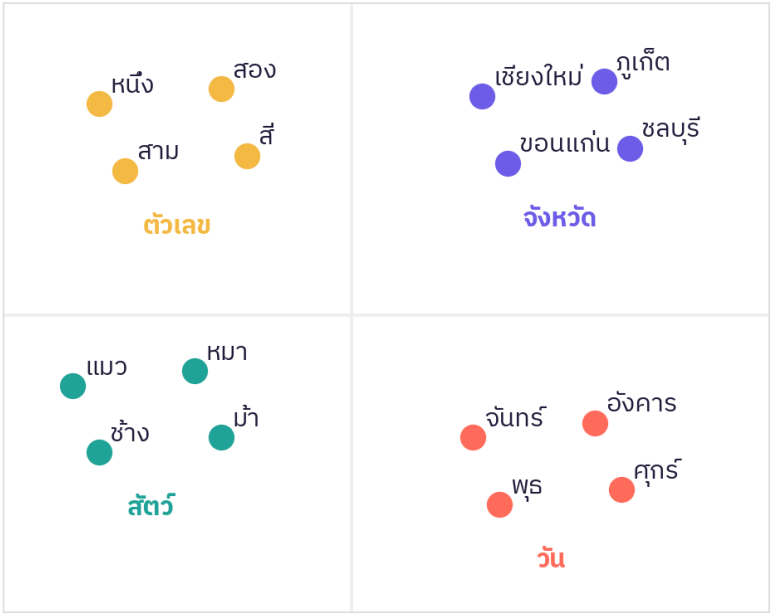
แปลง id ทีละตัวกลับเป็นข้อความ = ชิ้นที่ tokenizer ตัด

tokenizer ที่ฝึกภาษาไทยมา ใช้ token น้อยกว่ามาก = ประหยัด context และแต่ละ token มีความหมาย

Lab 1

Embedding: วาดให้เห็นกลุ่มคำ

Embedding: คำกลุ่มเดียวกัน อยู่ใกล้กัน
(ย่อจาก 2,048 มิติเหลือ 2 มิติ · ภาพตัวอย่าง)




```
vecs = [word_vector(w) for w in words]
# 20 คำไทย 4 กลุ่ม · 2048 มิติ → 2 มิติ
xy = PCA(n_components=2).fit_transform(vecs)
ax.scatter(xy[:,0], xy[:,1], c=กลุ่ม)
sim = cosine_similarity(ทุกคู่)    # 20×20
ax.imshow(sim)
```

สิ่งที่จะเห็น

คำกลุ่มเดียวกันอยู่ใกล้กัน (scatter) และเป็นบล็อกสว่างตามแนวทแยง (heatmap) — เรียนจากบริบทเอง ไม่มีใครบอก

LAB 2

ข้างใน LLM

จากตอนเช้า: Decoder เขียนทีละ token · Encoder vs Decoder · Base vs Instruction · Hallucination

เป้าหมาย

- ดู LLM เขียนทีละ token แบบเห็นทุกขั้น
- เทียบ attention จริงของ WangchanBERTa กับ Typhoon2
- เทียบ Base vs Instruct · มี/ไม่มีความจำ · มี/ไม่มี context

Lab 2

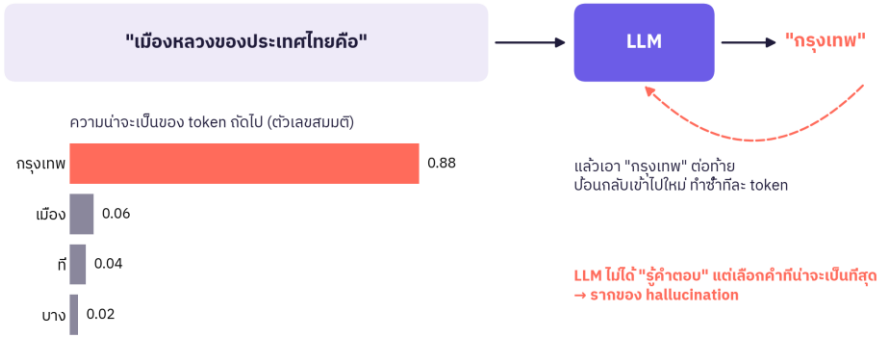
2.1 ข้อความ → id → โมเดล → id → ข้อความ




```
msgs = [{"role": "user", "content": "เมืองหลวงของญี่ปุ่นคืออะไร ตอบสั้นๆ"}]
inputs = tok.apply_chat_template(msgs, add_generation_prompt=True,
            return_tensors="pt", return_dict=True).to(device)          # ①
output_ids = model.generate(**inputs, max_new_tokens=30)               # ②
answer = tok.decode(output_ids[0][n_in:], skip_special_tokens=True)    # ③
```

Lab 2

2.2 ดู LLM "เขียน" ทีละ token



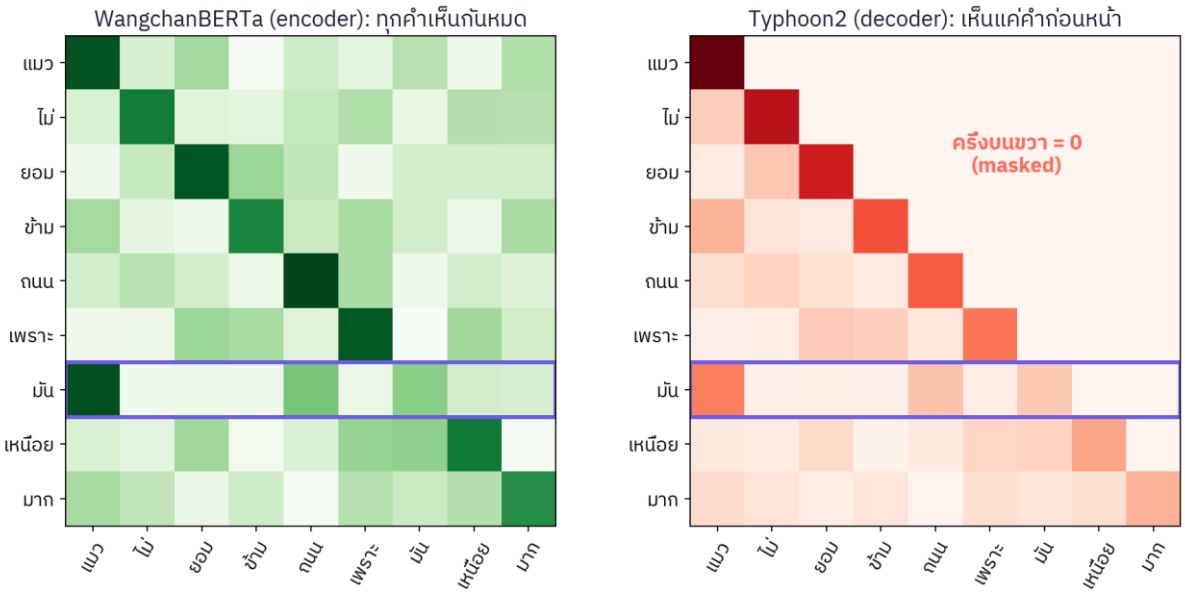

```
for s in range(6):
  logits = model(ids).logits
  probs = softmax(logits[0, -1])
  p, idx = topk(probs, 5)
  # กราฟแท่ง 5 อันดับ
  ids = cat([ids, idx[:1]])
  # ต่อคำที่ชนะ แล้ววนใหม่
```

สิ่งที่จะเห็น

6 กราฟเรียงกัน = 6 รอบของการเลือกคำ (autoregressive) บน Typhoon-Base · ดูว่า "กรุงเทพ" ชนะตั้งแต่รอบแรกไหม

Lab 2

2.3 Attention จริง: WangchanBERTa vs Typhoon2



แถว = คำที่กำลังคิด · คอลัมน์ = คำที่มองไป · กรอบม่วง = แถวของ "มัน" (ภาพจำลอง — ใน notebook ใช้ค่าจริงจากโมเดล)

```
out = model(**enc,
    output_attentions=True)
att =
out.attentions[layer][0]
att = att.mean(dim=0)   # เฉลี่ย
ทุก head
ax.imshow(att)
```

สิ่งที่จะเห็น

WangchanBERTa เต็มทุกช่อง · Typhoon2 เป็นสามเหลี่ยม (masked) · กราฟแยกแถว "มัน" ว่ามองไปที่ "แมว" ไหม

Lab 2

2.4 Base vs Instruct: ถามคำถามเดียวกัน



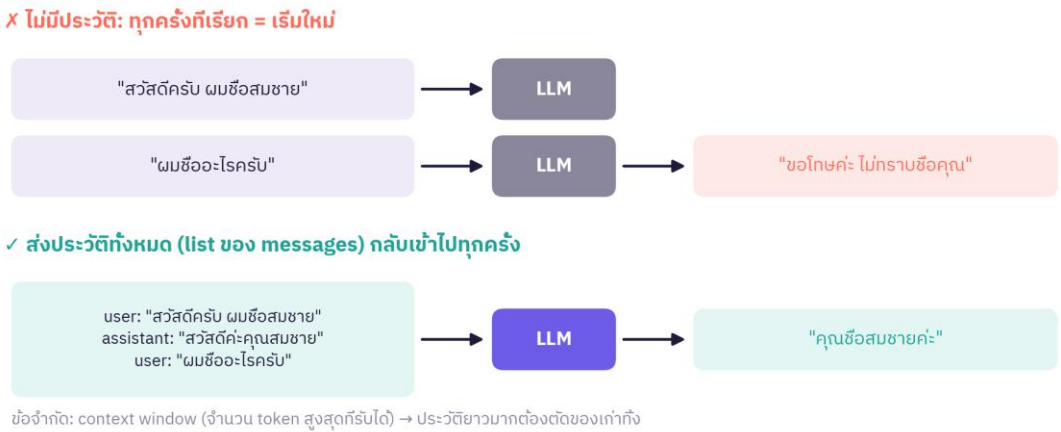

```
if name == INST:    # instruct
    inputs = tok.apply_chat_template(
                  msgs, ...)
else:               # base
    inputs = tok(prompt, ...)
out = mdl.generate(**inputs)
n = inputs["input_ids"].shape[1]
new_ids = out[0][n:]
```

compare(prompts)

ตาราง: 4 คำขอภาษาไทย (ทักทาย ข้อเท็จจริง แปล สรุป) × Base / Instruct

Lab 2

2.5 ความจำ: ไม่ส่งประวัติ vs ส่งประวัติ



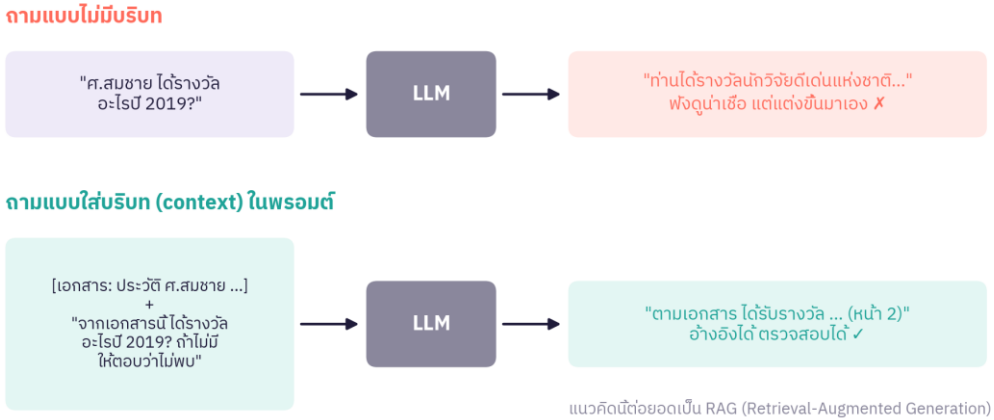

```
def chat(user_turns, memory):
  for text in user_turns:
    history.append({"role": "user",
                    "content": text})
    msgs = history if memory \
           else [history[-1]]
    reply = ask(INST, msgs)
```

ตาราง 3 คอลัมน์

ผู้ใช้พิมพ์ · ❌ ส่งแค่ข้อความล่าสุด · ✅ ส่งประวัติ

Lab 2

2.6 Hallucination: ไม่มี context vs มี context



คอลัมน์	มักจะเห็น
Base (เขียนต่อ)	แต่งต่อเสมอ ลื่นแต่ผิด
Instruct ไม่มี context	ตอบมั่นใจ อาจแต่งเอง
Instruct + context	ตรงข้อมูล / "ไม่พบข้อมูล"

ศ.สมชาย ใจดี เป็นบุคคลสมมติ · การฝึกต่างกัน → แนวโน้มแต่งคำตอบต่างกัน · context คือพื้นฐานของ RAG

LAB 3

Temperature & Top-p

จากตอนเช้า: LLM เลือกคำจากความน่าจะเป็น — ปุ่มสองตัวนี้ปรับการกระจาย "ก่อน" สุ่ม

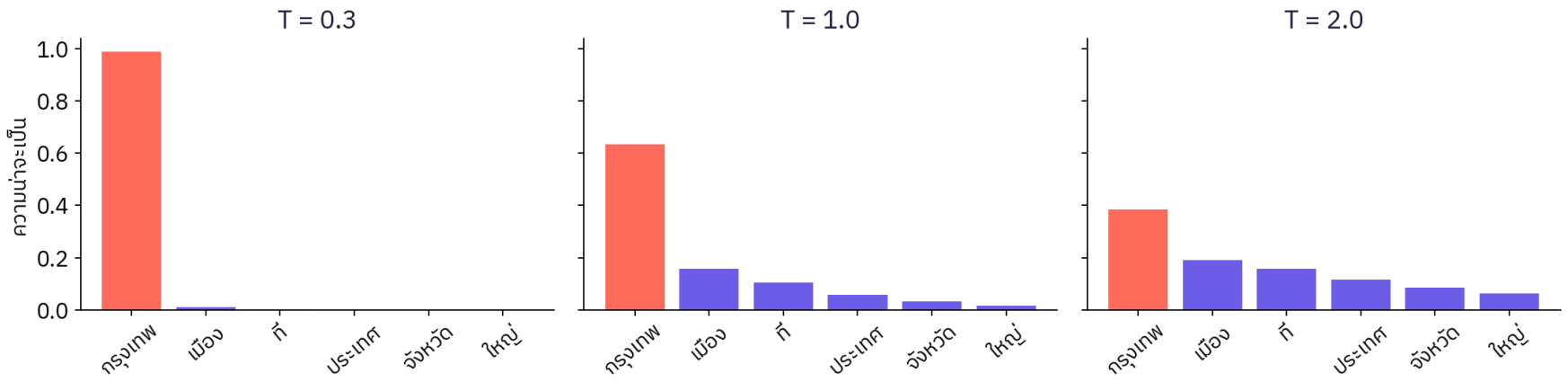
เป้าหมาย

- เห็นการกระจายจริงที่ temperature ต่างกัน
- เห็นว่า top-p เก็บตัวเลือกกี่คำในแต่ละสถานการณ์
- วัดความหลากหลายของข้อความ (distinct-2)

Lab 3

3.1 Temperature: softmax(logits / T)

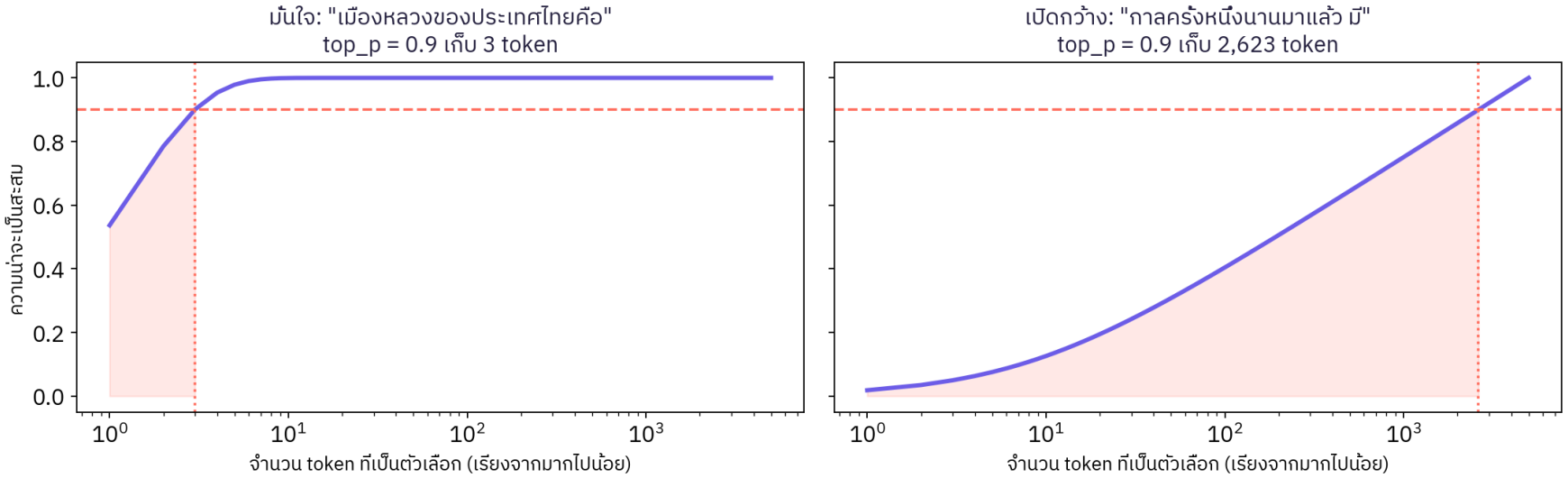
"เมืองหลวงของประเทศไทยคือ ___" (ตัวเลขสมมติเพื่ออธิบาย)




```
logits = next_token_logits("เมืองหลวงของประเทศไทยคือ")
for T in [0.3, 1.0, 2.0]:
    p = softmax(logits / T)    # T ต่ำ = แหลม · T สูง = แบน
```

Lab 3

3.2 Top-p: จำนวนตัวเลือกปรับตามความมั่นใจ



ภาพจำลองเพื่ออธิบาย — ใน notebook ใช้การกระจายจริงของ Typhoon2

พรอมต์ที่โมเดลมั่นใจ → เก็บไม่กี่คำ · พรอมต์เปิดกว้าง → เก็บหลายพัน token · ต่างจาก top_k ที่เก็บจำนวนตายตัว

Lab 3

3.3 ผลต่อข้อความจริง และค่าที่ควรใช้

พารามิเตอร์	ทำอะไร	ใช้เมื่อ
do_sample=False	greedy เลือกสูงสุดเสมอ	ข้อเท็จจริง/สรุป
temperature	ปรับความแหลม/แบน	0.2–0.3 แม่น · 0.7–1.0 สร้างสรรค์
top_p	สุ่มจากกลุ่มที่รวมถึง p	0.9–0.95
repetition_penalty	ลดการพูดซ้ำ	1.1–1.3

สิ่งที่จะเห็นใน notebook

ตาราง 5 ข้อความ × 3 temperature + กราฟ distinct-2: T สูงขึ้น = หลากหลายขึ้น แต่เริ่มหลุด

จำไว้

temperature ต่ำ = คงที่ แต่ไม่ได้กัน hallucination

LAB 4

VAE vs Autoencoder

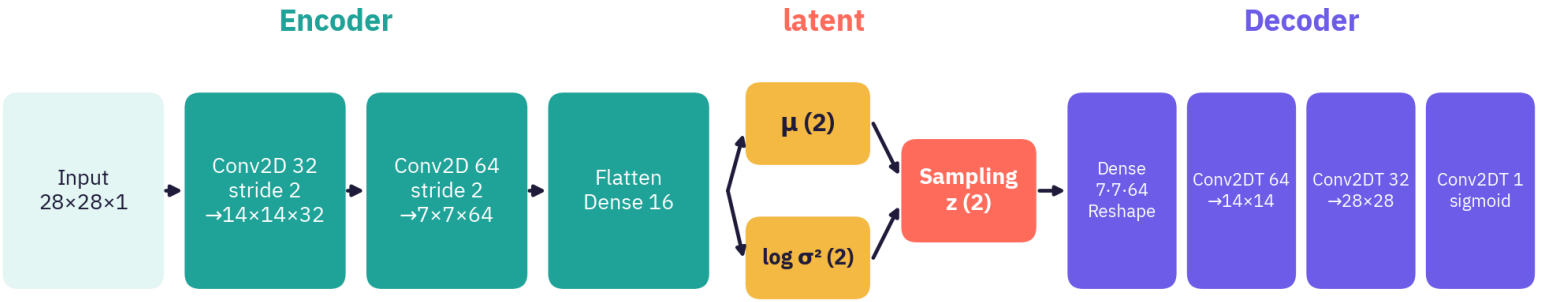
จากตอนเช้า: Autoencoder บีบเป็น "จุด" · VAE บีบเป็น "ช่วง" จึงสุ่มสร้างได้

เป้าหมาย

- เทรน 2 โมเดลด้วยโค้ดเดียวกัน ต่างแค่ kl_weight
- เทียบ 5 อย่าง: loss · reconstruction · latent · แผนที่ · การสุ่มสร้าง
- เข้าใจว่าเทอม KL ทำให้ VAE เป็น generative model

Lab 4

สถาปัตยกรรม: Encoder → Sampling → Decoder



Loss = Reconstruction (ภาพออกเหมือนภาพเข้า) + KL (บังคับ z ให้ใกล้ N(0,1))

```
z = z_mean + ops.exp(0.5 * z_log_var) * epsilon      # reparameterization trick
```

สวิตช์เดียว: kl_weight

```
recon_loss = mean(sum(binary_crossentropy(data, recon)))
kl = -0.5 * (1 + z_log_var - square(z_mean) - exp(z_log_var))
kl_loss = mean(sum(kl, axis=1))
loss = recon_loss + self.kl_weight * kl_loss

vae, _ = train(1.0, "VAE")           # มี KL
ae,  _ = train(0.0, "Autoencoder")   # ไม่มี KL
```

kl_weight = 1

VAE: ดึง latent เข้าหา N(0,1)

kl_weight = 0

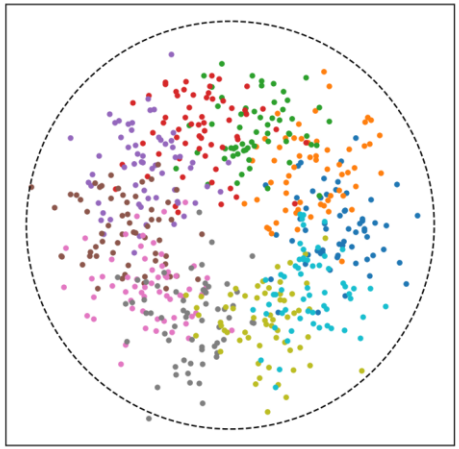
Autoencoder ธรรมดา: ไม่มีอะไรคุม latent

ทดลองแบบควบคุมตัวแปร: สถาปัตยกรรม ข้อมูล จำนวน epoch เหมือนกันทุกอย่าง ต่างแค่เทอมเดียว

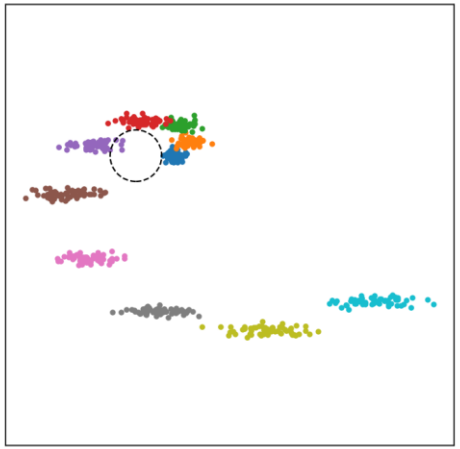
Lab 4

ผลที่จะเห็น: ลอกภาพได้ทั้งคู่ แต่สร้างใหม่ได้แค่ VAE

VAE: อัดแน่นรอบ (0,0) ต่อเนื่อง



Autoencoder: กระจาย มีช่องว่าง



วงประ = บริเวณที่การสุ่ม z ~ N(0,1) ตกบ่อย → VAE มีข้อมูลเต็มวง จึงสุ่มสร้างได้ · ภาพจำลองเพื่ออธิบาย

เทียบ	AE	VAE
Reconstruction	ดี	ดี (เบลอกว่านิด)
Latent	กระจาย มีช่องว่าง	อัดแน่นรอบ 0
แผนที่ [-2,2]	ภาพซ้ำ/เละ	ค่อยๆ เปลี่ยน
สุ่ม z ~ N(0,1)	ไม่น่าเชื่อถือ	ได้ตัวเลขใหม่
Interpolation	สะดุด	ลื่นไหล

ความต่างทั้งหมดมาจากเทอม KL เทอมเดียว = เหตุผลที่ VAE เป็น generative model

การบ้าน

การบ้าน: ระบบวิเคราะห์รีวิวภาษาไทย (Notebook 3)

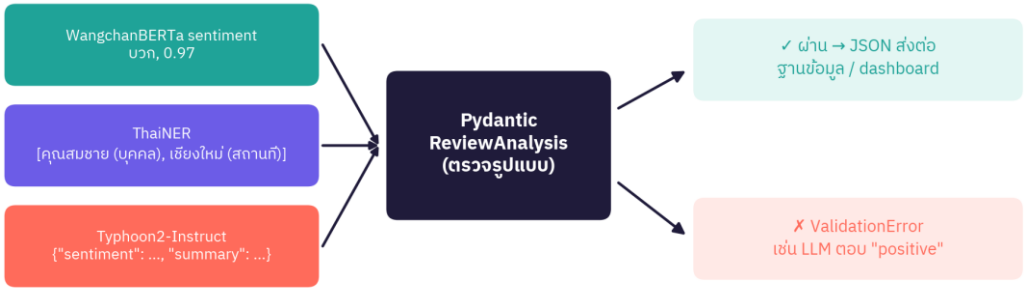
ร้านกาแฟสมมติ ดอยดาว คาเฟ่ ต้องการระบบอ่านรีวิวภาษาไทย → ข้อมูลที่ตรวจสอบแล้ว + กราฟสรุปให้ผู้บริหาร

งาน	เนื้อหา	คะแนน
1	Pydantic model (ผล LLM + ผลรวม) + วิเคราะห์ 12 รีวิวเป็นตาราง	3
2	ภาพสรุป 3 กราฟ ป้ายภาษาไทย (สินค้า×ความรู้สึก, confidence, ชื่อเฉพาะ)	3
3	WangchanBERTa vs Typhoon2 LLM (heatmap + อัตรา JSON ถูกรูปแบบ)	2
4	ส่งออก JSON ที่ตรวจแล้ว + จับข้อมูลผิดรูปแบบ	1
5	บทวิเคราะห์ข้อจำกัด 150–250 คำ	1

ส่ง: ไฟล์ .ipynb ที่รันครบทุกเซลล์ (มีผลลัพธ์และกราฟ) · File → Download → .ipynb

การบ้าน

เครื่องมือในการบ้าน และสิ่งที่ควรสังเกต



เครื่องมือ	ตระกูล
sentiment_a (WangchanBERTa)	encoder + head
thai_ner (ThaiNER v2)	encoder + token head
llm_review (Typhoon2)	decoder · instruct

ข้อมูลมีรีวิวผสม · ประชด · ข้อเท็จจริงล้วน

เพื่อให้เห็นว่า confidence สูงไม่ได้แปลว่าถูก · LLM เล็กบางครั้งตอบผิดรูปแบบ → Pydantic จับได้ แต่ตรวจความถูกต้องของเนื้อหาไม่ได้

สรุป

สิ่งที่ทำบ่ายนี้ ↔ ทฤษฎีช่วงเช้า

บ่ายนี้ (เห็นด้วยตา)	ทฤษฎีช่วงเช้า
token สี: tokenizer อังกฤษ vs ไทย, กลุ่มคำไทยใน embedding	Token → Vector
กราฟ 6 รอบของการเลือกคำ	Decoder เขียนทีละ token
WangchanBERTa เต็ม vs Typhoon2 สามเหลี่ยม	Encoder vs Decoder, masked attention
Typhoon-Base vs Typhoon-Instruct	Base vs Instruction, ฝึก LLM เป็นขั้น
ความจำ, context ลดการแต่ง	Context window, Hallucination → RAG
temperature, top-p, distinct-2	การเลือกคำจากความน่าจะเป็น
VAE vs AE: เทอม KL	Latent space, VAE
การบ้าน: โมเดลเฉพาะงาน vs LLM + Pydantic	Body + head, Instruction model